

7 适配器和外观模式

★ 随遇而安 ★

你认为读者真的觉得我们在看赛马，而不是坐在照相馆里？

这就是我们专业的魅力：我们可以让一个东西看起来像其他东西！

你的意思是说，这不应该是一场足球赛？

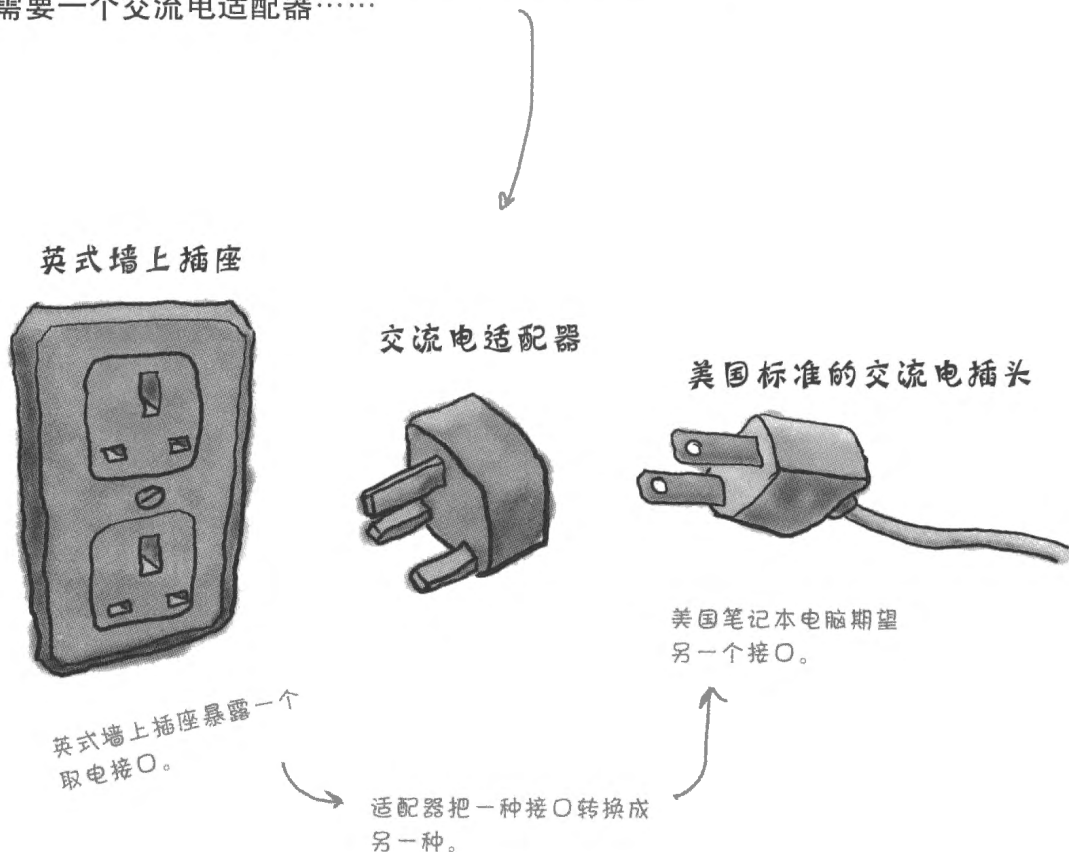
包装在这件外套里，我就变成了一个完全不同的人！



本章中，我们打算尝试不可能的任务，相当于把方形桩放入圆孔。听起来不可能？那是我们没有设计模式的时候。还记得装饰者模式吗？我们包装对象，赋予它们新的责任。现在，我们打算为不同的目的包装一些对象，使它们的接口看起来像别的东西。为什么我们要这样做？因为这样我们就可以把期望某个接口的设计适配到一个实现不同接口的类。不仅如此，我们还打算趁热打铁，探讨另一个包装对象以简化其接口的模式。

我们周围的适配器

理解什么是OO适配器，对你来说应该没什么问题，因为现实世界到处都是。例如：你是否曾经需要在英国使用美国制造的笔记本电脑？那么你可能需要一个交流电适配器……



你知道适配器做什么，它位于你的笔记本电脑插头和英式交流电插座之间，其工作是适配英式插座，这样你可以把你的笔记本电脑插入并接收电力。或者可以这样看，适配器改变了插座的接口，以符合你的笔记本电脑的期望。

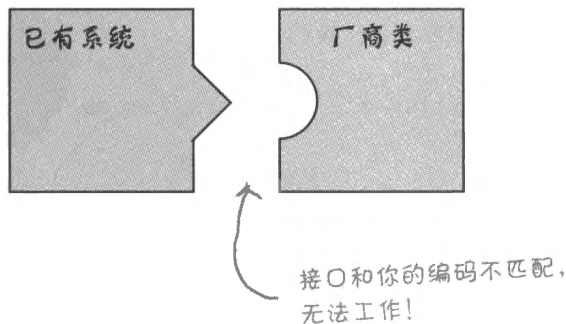
有些交流电适配器很简单，只是改变插座的形状以便匹配插座，它们直接传递交流电—但其他适配器内部更复杂，可能需要增加或降低电压以匹配设备的需要。

OK，这是现实世界，那面向对象适配器呢？嗯，我们的OO适配器扮演和现实世界同样的角色，把一个接口适配成符合客户期望的接口。

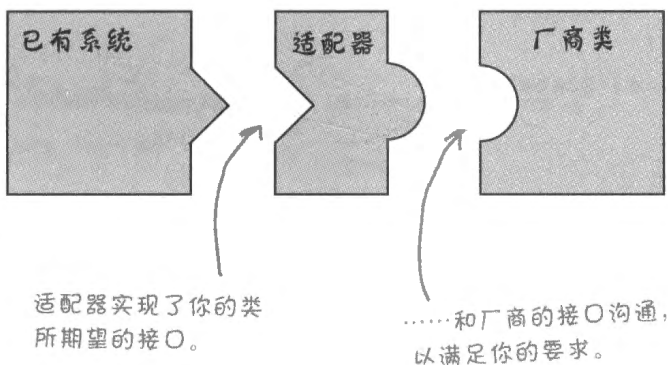
你能想到现实世界中，还有哪些适配器？

面向对象适配器

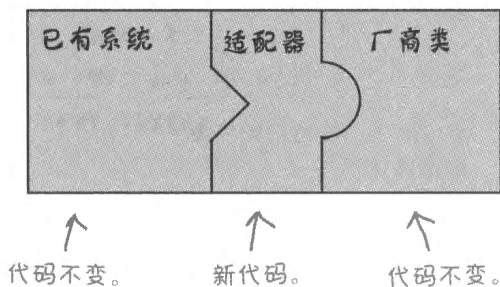
假设已有一个软件系统，你需要把一个新的厂商类库整合进去，但新的厂商设计的接口和之前的厂商不同。



OK，你不想通过改变已有代码来解决问题（而且你不能改变厂商的代码）。因此你要怎么做？嗯，你可以编写一个类，把新的厂商接口适配成你所期望的接口。



适配器扮演中间人。它接收来自客户的请求，并转换成厂商类能理解的请求。



你能想到一个解决方案，不用写任何额外的代码，就可以集成新的厂商类吗？让厂商提供这个适配器类怎么样？

如果它走路像鸭子，叫起来像鸭子，那么它一定可能是鸭子被鸭子适配器包装的火鸡……

是时候看看适配器如何工作了。还记得第1章我们的鸭子吧？我们来看一个稍微简化的版本的Duck接口和类：

```
public interface Duck {
    public void quack();
    public void fly();
}
```

这一次，我们的鸭子实现了Duck接口，使得鸭子可以呱呱叫和飞行。



这是一个Duck的子类，MallardDuck（绿头鸭）：

```
public class MallardDuck implements Duck {
    public void quack() {
        System.out.println("Quack");
    }

    public void fly() {
        System.out.println("I'm flying");
    }
}
```

简单实现：MallardDuck只是打印出正在做的事情。

现在，是时候认识最新的街头猛禽了：

```
public interface Turkey {
    public void gobble();
    public void fly();
}
```

火鸡不呱呱叫，它们咯咯叫（gobble）。

火鸡能飞，虽然它们只能短距离飞行。

```

public class WildTurkey implements Turkey {
    public void gobble() {
        System.out.println("Gobble gobble");
    }

    public void fly() {
        System.out.println("I'm flying a short distance");
    }
}

```

这是Turkey的一个具体实现；
像MallardDuck一样，只是打印
出其动作。

现在，假设你的Duck对象短缺，想用一些Turkey对象来充数。显然我们不能堂而皇之地使用火鸡，因为它们接口不同。

因此，我们来编写一个适配器：



Code up Close

首先，你需要实现要适配类型的接口。
这是你的客户期望看到的接口。

```

public class TurkeyAdapter implements Duck {

```

```

    Turkey turkey;

```

```

    public TurkeyAdapter(Turkey turkey) {

```

```

        this.turkey = turkey;
    }

```

```

    public void quack() {

```

```

        turkey.gobble();
    }

```

```

    public void fly() {

```

```

        for(int i=0; i < 5; i++) {

```

```

            turkey.fly();
        }
    }
}

```

接下来，我们需要取得要适配的对象引用，这里我们通过构造器得到。

现在我们需要实现接口中的所有方法，quack()在类之间的翻译很容易，只要调用gobble()方法。

即使两个接口都有fly()方法，火鸡的飞行只是短距离冲刺，不能像鸭子那样长途飞行。要在Duck的fly()方法和Turkey的fly()方法之间形成映射，我们需要调用Turkey的fly()方法五次来补足。

测试适配器

现在我们只需要一些代码来测试适配器：

```
public class DuckTestDrive {
    public static void main(String[] args) {
        Duck duck = new MallardDuck();

        Turkey turkey = new WildTurkey();
        Duck turkeyAdapter = new TurkeyAdapter(turkey);

        System.out.println("The Turkey says...");
        turkey.gobble();
        turkey.fly();

        System.out.println("\nThe Duck says...");
        testDuck(duck);

        System.out.println("\nThe TurkeyAdapter says...");
        testDuck(turkeyAdapter);
    }

    static void testDuck(Duck duck) {
        duck.quack();
        duck.fly();
    }
}
```

创建一只Duck……

……和一只Turkey。

然后把火鸡包装进一个TurkeyAdapter，这让它看起来像是一只Duck。

然后，测试Turkey，让它咯咯叫，让它飞。

现在测试鸭子，调用testDuck()方法，需要传入一个Duck对象。

现在来大测试：我们试着传入一只假装是鸭子的火鸡……

这是我们的testDuck()方法：它取得一只鸭子，并调用其quack()和fly()方法。

测试结果

```
File Edit Window Help Don'tForgetToDuck
%java DuckTestDrive
The Turkey says...
Gobble gobble
I'm flying a short distance

The Duck says...
Quack
I'm flying

The TurkeyAdapter says...
Gobble gobble
I'm flying a short distance
I'm flying a short distance
I'm flying a short distance
I'm flying a short distance
I'm flying a short distance
```

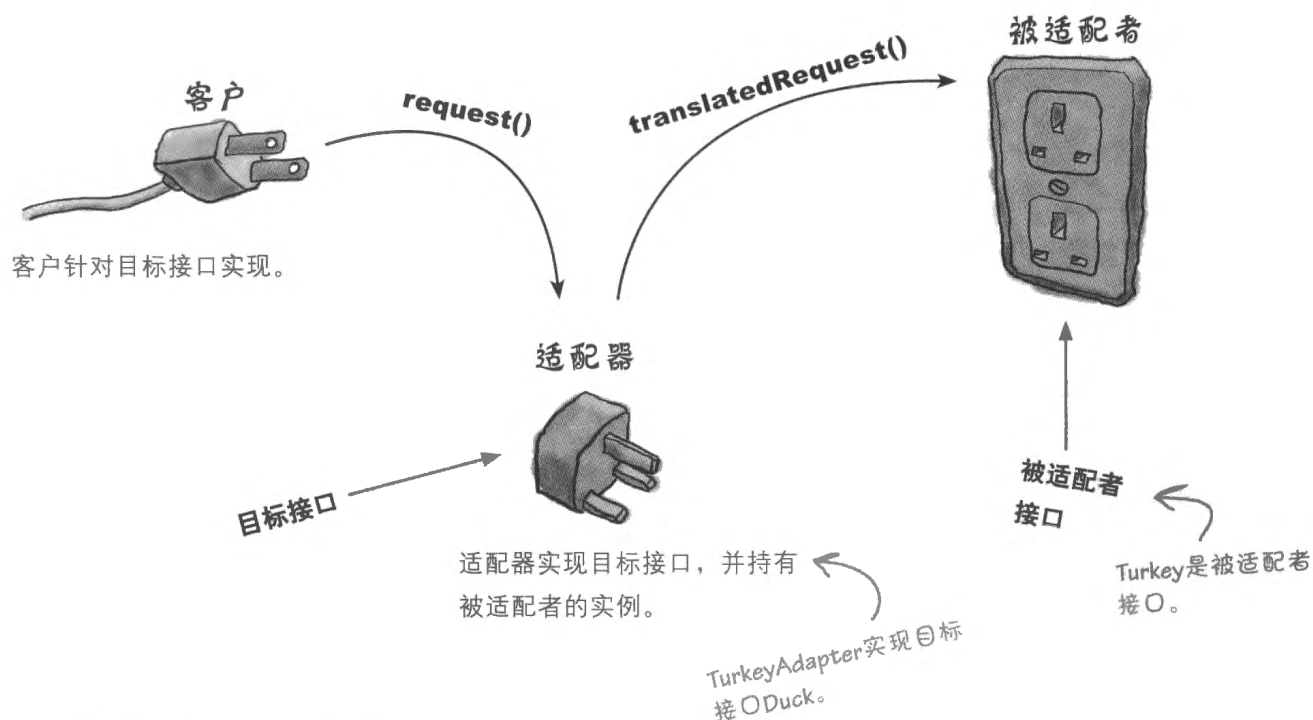
火鸡咯咯叫，且飞行距离短。

鸭子呱呱叫，并像你期望的那样飞行。

quack()被调用时，适配器咯咯叫；fly()被调用时，适配器飞行多次。testDuck()方法根本不知道，这其实是一只假装成鸭子的火鸡！

适配器模式解析

现在我们知道什么是适配器了，让我们后退一步，再看看所有片段。



客户使用适配器的做法如下

- 1 客户通过使用目标接口，调用适配器的方法，对适配器做出请求。
- 2 适配器使用被适配者接口，把请求翻译成被适配者上的一个或多个调用。
- 3 客户收到调用结果，但根本不知道是适配器在做翻译。

注意，客户和被适配者解耦了，彼此不知道对方。



削尖你的铅笔

假设我们也需要一个适配器，将鸭子转成火鸡。我们称它为 DuckAdapter。请编写这个类：

你如何处理 fly() 方法（毕竟我们知道鸭子飞得比火鸡远）？答案在本章最后。你想得到更好的方法吗？

没有蠢问题

问：适配器需要做多少次“适配”？看起来如果我需要实现一个大的目标接口，有大量工作要做。

答：确实如此。实现一个适配器的的工作，真的和你需要支持的目标接口大小成正比。可以考虑一下如何选择。你可以重写所有客户端对接口的调用，这会造成大量调查工作和代码变更。或者，你可以干脆利落地提供一个类，在这个类中封装所有变化。

问：适配器总是只能包装一个类吗？

答：适配器模式的角色是转化一个接口到另一个。大多数适配器模式例子展示的是适配器包装一个被适配者。我们都知道，这个世界经常要更杂乱一些。因此，就会有一个适配器持有两个或以上要实现目标接口的被适配者的情况。

这和另一个叫作外观（Facade）的模式有关，人们经常搞混两者。本章后面，我们谈到外观时，还会再提到这一点。

问：要是我的系统中新旧并存，旧的部件期望旧的厂商接口，但我们已经编写新的部件来使用新的厂商接口，这怎么办？这里使用适配器，那里却使用未包装的接口，这会让人感到混淆。如果我只是编写旧的代码，忘掉适配器，会不会更好？

答：不必如此。你可以创建一个支持两个接口的双向适配器。为了创建双向适配器，要实现涉及的两个接口，因此，适配器可以扮演老的接口，或者新的接口。

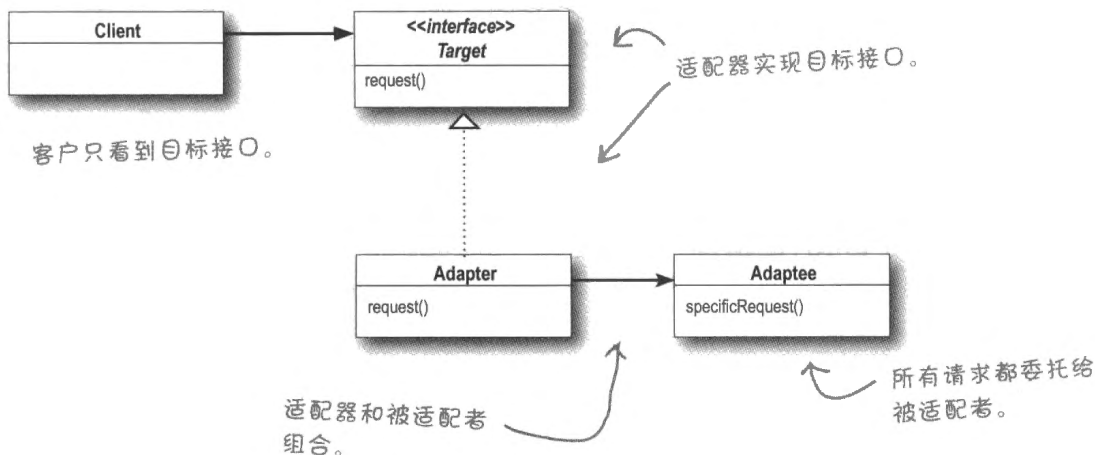
定义适配器模式

看够了鸭子、火鸡和交流电适配器，我们进入现实世界，看看适配器模式的官方定义：

适配器模式 将一个类的接口转换成客户期望的另一个接口。适配器让原本接口不兼容的类可以合作。

现在，我们知道，通过创建一个适配器来做转换，这个模式允许我们使用带有不兼容接口的客户。这使得客户从所实现的接口解耦，如果我们预料到接口会随着时间的推移而变化，适配器可以封装变化，这样，每一次需要针对不同的接口操作时，客户不必都要修改。

我们已经看过该模式的运行时行为；我们也来看看它的类图：



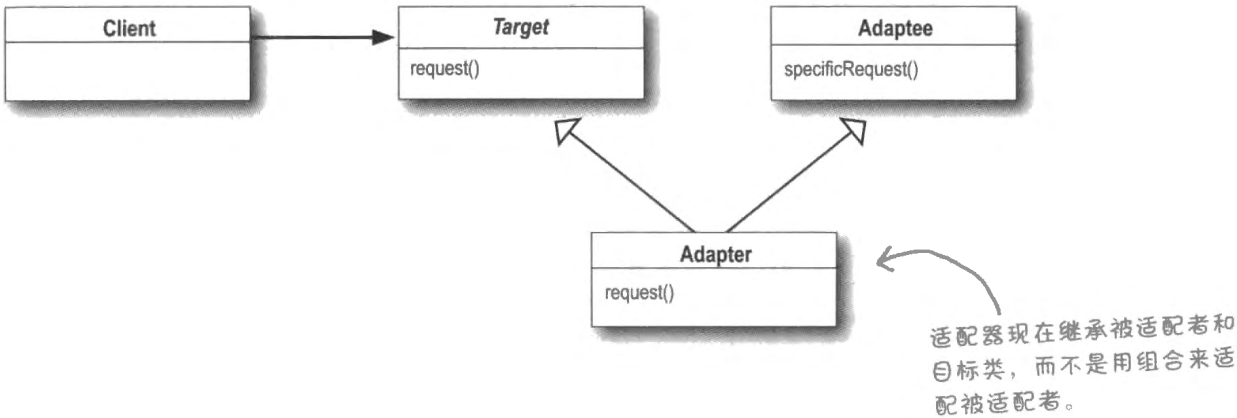
适配器模式里充满了良好的面向对象设计原则：可以看到对象组合的使用，用修改的接口来包装被适配者。这个做法还有额外的优点，我们可以使用适配器搭配被适配者的任何子类。

还可以看到，模式如何把客户和接口绑定，而不是和实现绑定。我们可以使用若干适配器，每一个转化不同的后端类集。或者，我们可以在事后添加新的实现，只要它们遵从目标接口。

对象和类适配器

尽管现在已经定义了这个模式，我们还没有告诉你整个故事。实际上有两种适配器：对象适配器和类适配器。本章涵盖了对象适配器，上一页的类图就是对象适配器的图。

那什么是类适配器？为什么我们还没有告诉你关于它的故事？因为实现它需要多重继承，这在Java中是不可能的。但是往下走，当你使用喜爱的多重继承语言时，你可能会面临需要类适配器的情况！我们来看看多重继承的类图。



看着熟悉？没错，唯一的区别是用了类适配器。我们继承了Target和Adaptee，而对象适配器用组合把请求传递给Adaptee。



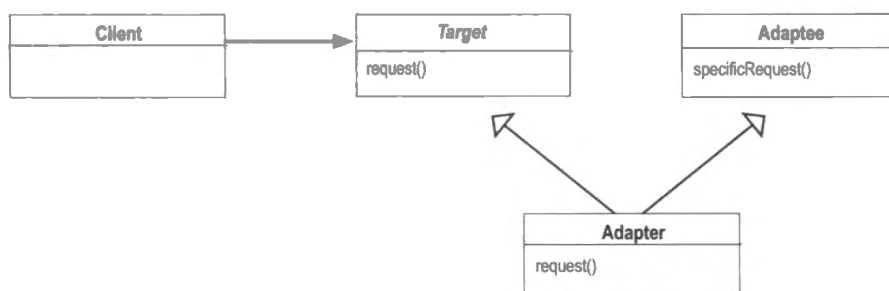
对象适配器和类适配器使用两种不同的适配方法（组合与继承）。这两种实现的差异如何影响适配器的弹性？



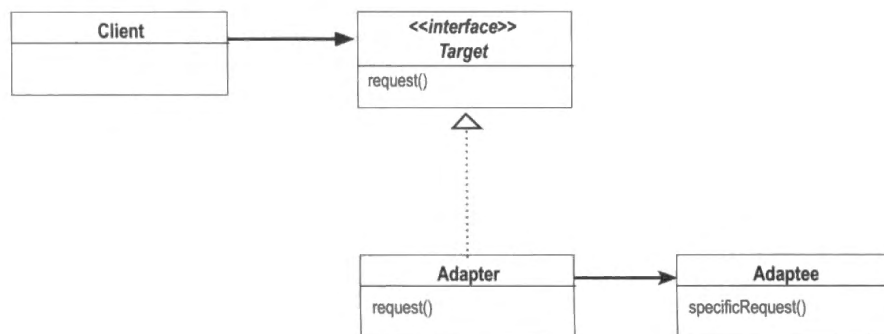
鸭子磁贴

你的工作是把鸭子和火鸡的磁贴拖动到下图所描述的之前例子的角色上（试着不要翻页看）。然后添加你自己的注解来描述其如何工作。

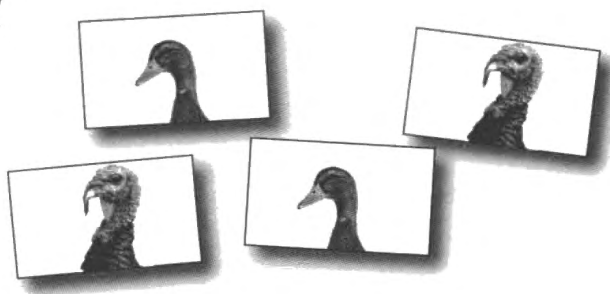
类适配器



对象适配器



把这些拖到类图上，展示图中的哪一部分代表Duck类，哪一部分代表Turkey类。

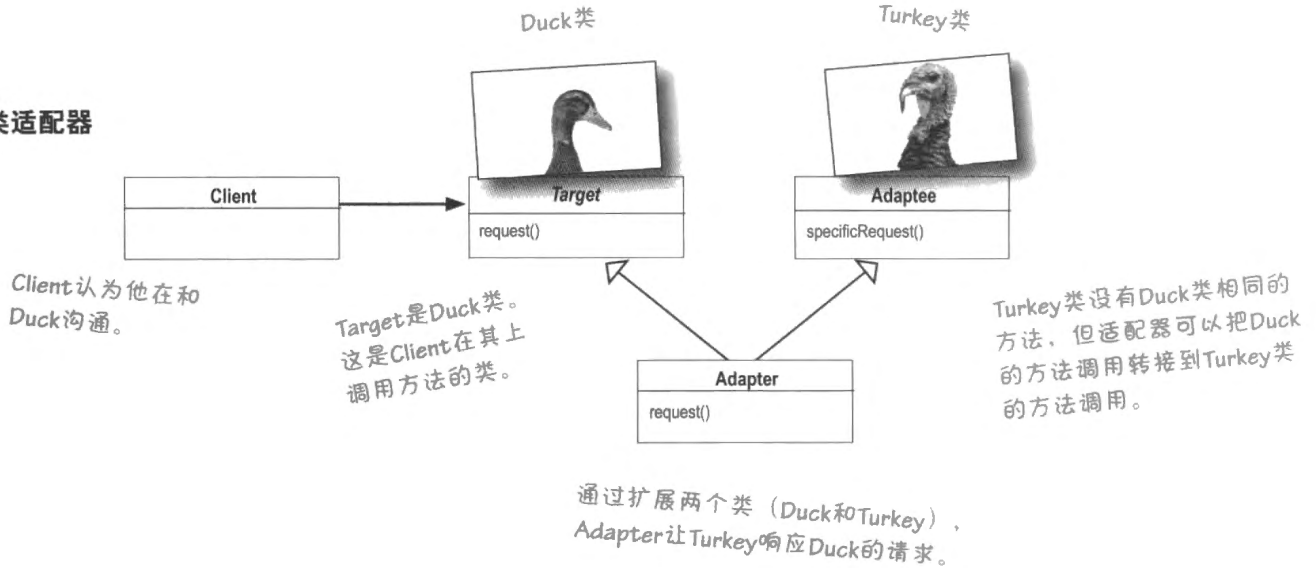




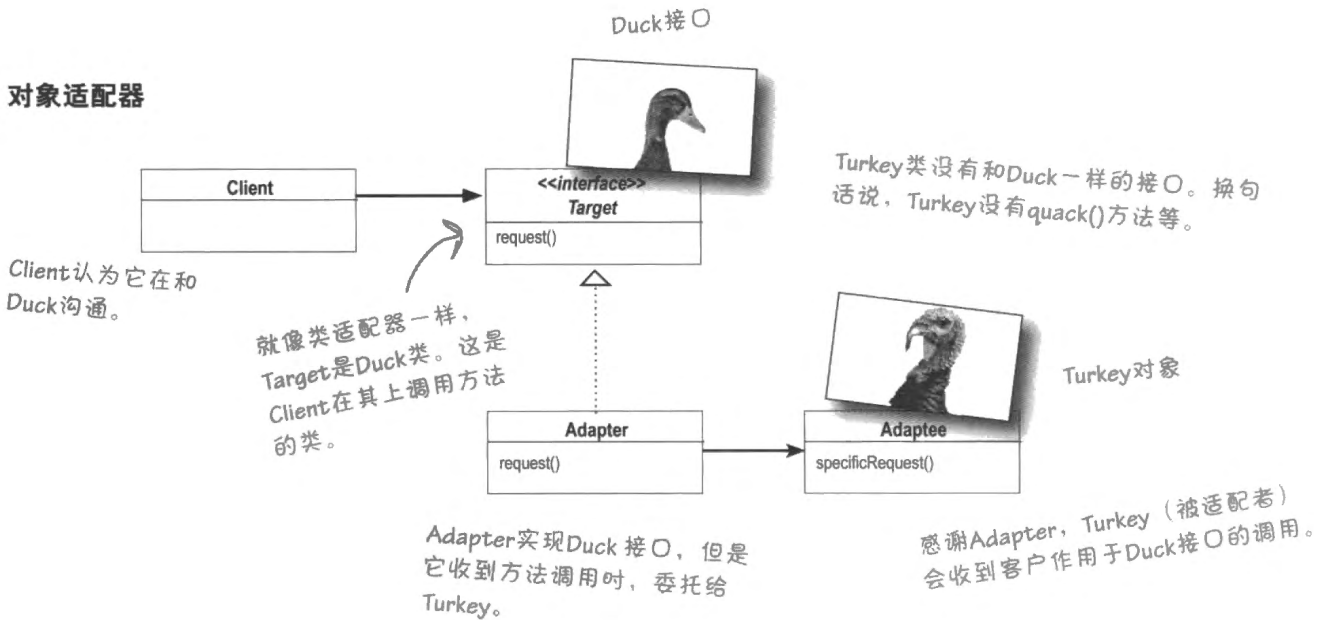
鸭子磁贴答案

注：类适配器使用多重继承，因此你在Java中做不了……

类适配器



对象适配器



炉边谈话



今夜话题：对象适配器和类适配器
面对面。

对象适配器：

因为使用组合，所以更胜一筹。我不但可以适配一个被适配者类，还可以适配它的任何子类。

在我那部分的世界中，我们更喜欢使用组合而不是继承；我只需要写一点点代码，将工作委托给被适配者。我们喜欢让事情保持弹性。

你担心一个小小的对象？你也许能够很快覆盖一个方法，但我添加到适配器代码中的任何行为，都可以和我的被适配者类以及它的所有子类搭配运作。

咳，拜托，放我一马吧，我只需要和子类组合来解决问题。

你想看看什么是乱糟糟吗？看看镜子！

类适配器：

确实，我做不到这一点，因为我只针对一个特定的被适配者类，但我有一个巨大的优势，我不必重新实现整个被适配者。需要时，我也可以覆盖被适配者的行为，因为我使用子类化。

有弹性，也许吧，但效率呢？没有了吧。我只需要一个，而不是一个适配器搭配一个被适配者。

是的，但如果被适配者的子类添加一些新行为——会怎样？

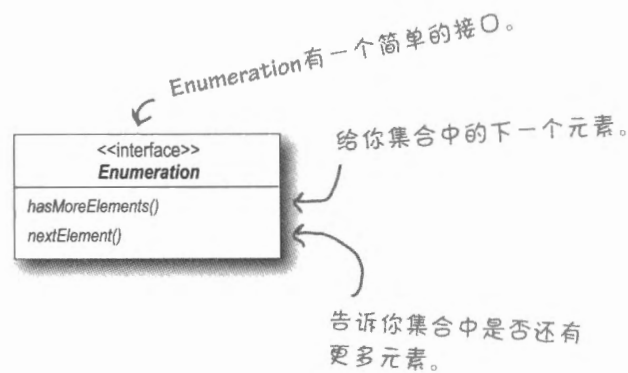
听起来乱糟糟的……

现实世界的适配器

我们看看简单适配器在现实世界中的使用（至少比鸭子要严肃一些）……

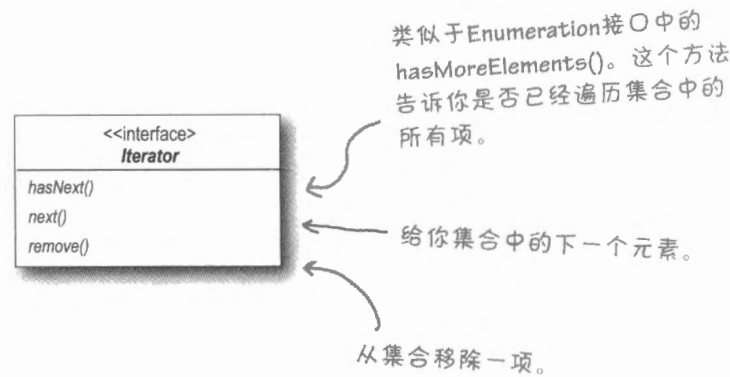
枚举器

如果用过一段时间Java，你可能记得，早期的集合类型（Vector、Stack、Hashtable及其他）都实现了一个方法，elements()，该方法返回一个Enumeration（枚举）。Enumeration接口能让你遍历集合的元素，而无需知道它们在集合内是如何被管理的。



迭代器

更新的集合使用Iterator（迭代器）接口，它类似于Enumeration接口，能让你遍历集合内的每一项，还添加了移除集合项的能力。

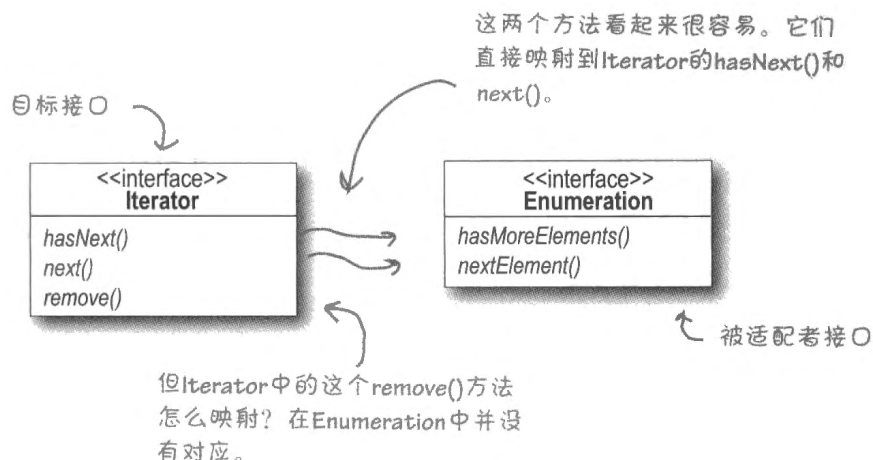


和期盼Iterator的代码一起使用枚举器

我们有时会面对暴露Enumeration接口的遗留代码，而我们还是喜欢让新代码只使用Iterator。看起来，我们需要建造一个适配器。

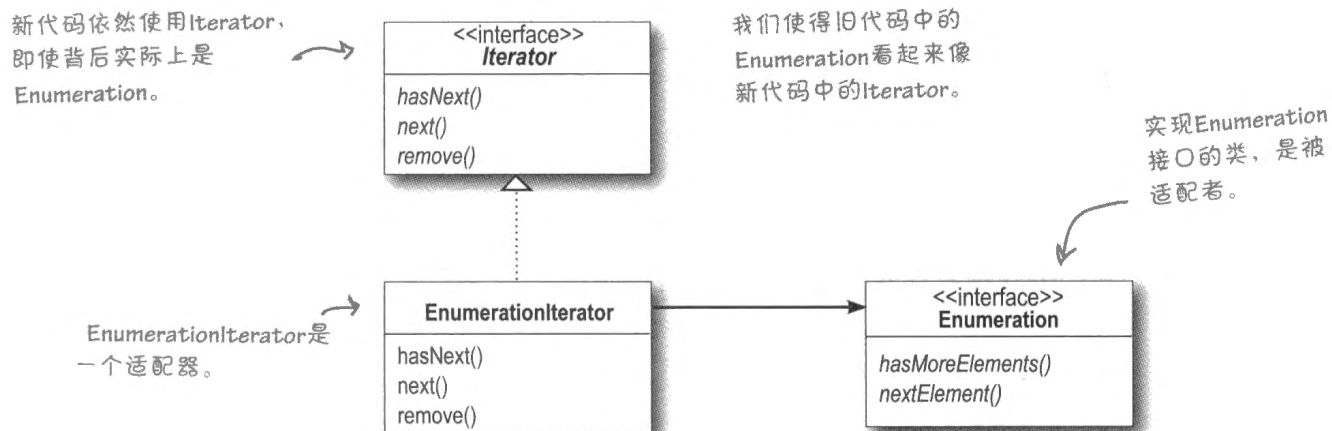
适配 Enumeration 到 Iterator

先看看这两个接口，弄清楚方法如何从一个映射到另一个。换句话说，当客户调用目标的方法时，我们要找到被适配者上要调用的方法。



设计适配器

这个类应该看起来像这样：我们需要一个实现目标接口并和被适配者组合的适配器。hasNext() 和 next() 方法直接从目标到被适配者映射即可，我们只是直接传递。但 remove() 怎么办？考虑一会（我们在下一页就会处理）。目前，类图如下：



处理remove()方法

我们知道Enumeration不支持remove()。它是一个“只读”接口。适配器无法实现一个有完全功能的remove()方法。我们可以做得最好情况是抛出一个运行时异常。幸运的是，Iterator接口的设计师预见了这个需要，定义remove()方法为支持一个UnsupportedOperationException。

这是一个适配器不完美的例子，客户不得不小心潜在的异常，但只要客户小心在意，而且适配器的文档良好，这是一个相当合理的解决方案。

编写EnumerationIterator适配器

这是一份简单而高效的代码，适合所有依然会生产Enumeration的遗留类：

```
public class EnumerationIterator implements Iterator<Object> {
    Enumeration<?> enumeration;

    public EnumerationIterator(Enumeration<?> enumeration) {
        this.enumeration = enumeration;
    }

    public boolean hasNext() {
        return enumeration.hasMoreElements();
    }

    public Object next() {
        return enumeration.nextElement();
    }

    public void remove() {
        throw new UnsupportedOperationException();
    }
}
```

因为我们把Enumeration适配成Iterator，所以适配器实现Iterator接口……它必须看起来像一个Iterator。

我们正在适配的Enumeration。我们使用组合，因此我们把它存放在一个实例变量中。

Iterator的hasNext()方法被委托给Enumeration的hasMoreElements()方法……

……Iterator的next()方法被委托给Enumeration的nextElement()方法。

不幸的是，我们不能支持Iterator的remove()方法，因此不得不放弃。在这里，我们只是抛出一个异常。



虽然Java已经走在Iterator接口的方向上，但仍然有依赖于Enumeration接口的遗留客户代码，因此一个把Iterator转换成Enumeration的适配器很有用。

编写一个适配器把Iterator适配到Enumeration。可以通过适配一个ArrayList来测试你的代码。ArrayList类支持Iterator接口，但不支持Enumeration。



某些交流电适配器所做的事情不只是改变接口，它们还添加了一些其他的特性，例如：电涌保护、指示灯和警铃警哨。

如果要你实现这类特性，你要使用什么模式？

炉边谈话



今夜话题：装饰者模式和适配器模式讨论彼此差别。

装饰者：

我很重要。我的工作都是关于“责任”的——你知道，当装饰者被涉及时，就表示有一些新的责任或行为要添加到设计中。

这可能是真的，但不要认为我们工作不努力。当我们不得不装饰一个大接口时，哇，可是需要很多代码的。

耍滑头。别认为我们独揽所有的光环，有时候我只是一个不知被多少其他装饰者包装的装饰者。当一个方法调用委托给你时，你根本不知道有多少其他装饰者已经处理过它，你不知道，为了服务这个请求所做的努力是否会得到注意。

适配器：

你们装饰者想要所有的荣耀，而我们适配器却俯身于沟渠中干着脏活：转换接口。我们的工作可能不光彩夺目，但客户肯定感激我们让他们的生活更简单。

当你必须将若干类整合在一起，来提供你的客户所期望的接口时，如果是适配器就麻烦了。不过，我们有一句格言：“被解耦的客户才是快乐的客户。”

嘿，如果适配器正在做它们的工作，客户绝不会知道我们在那。这个工作吃力不讨好。

装饰者：

嗯，我们装饰者也可以做到，只有我们允许新行为添加到类中而无需修改已有代码。我还是要说，适配器只是一种别致的装饰者。我的意思是，就像我们一样，包装一个对象。

唔，不是。我们的工作扩展所包装对象的行为或责任，不是简单的传递。

也许我们应该各自保留意见。我们在纸面上看起来有些类似，但很显然，我们的意图差别很大。

适配器：

但是，适配器的最大好处是，我们允许客户使用新的库和子集，无须改变任何代码；它们只需依靠我们来做转换即可。嘿，这是一个利基，但我们擅长这个。

不，不，不，根本不是。我们总是转化所包装的接口，你们绝不会。我要说，装饰者像适配器，只不过你们不改变接口！

嘿，你调用谁简单传送？来，你来转化一些接口，我看看你能撑多久！

没错，这一点我同意。

谁做什么？

现在，看看不同之处……

本章还有另一个模式。

你已经看到适配器模式把类的接口转化成客户期望的接口。你也知道，在Java中，我们达到目的的做法是，把一个有不兼容接口的对象用实现正确接口的对象包装起来。

现在，我们打算看一个模式，它改变接口的原因不一样：为了简化接口。它被恰当地命名为外观（Facade）模式，因为这个模式把一个或多个类所有的复杂性隐藏在一个整洁明亮的外观背后。

★ WHO DOES WHAT? ★

把每个模式和其意图匹配：

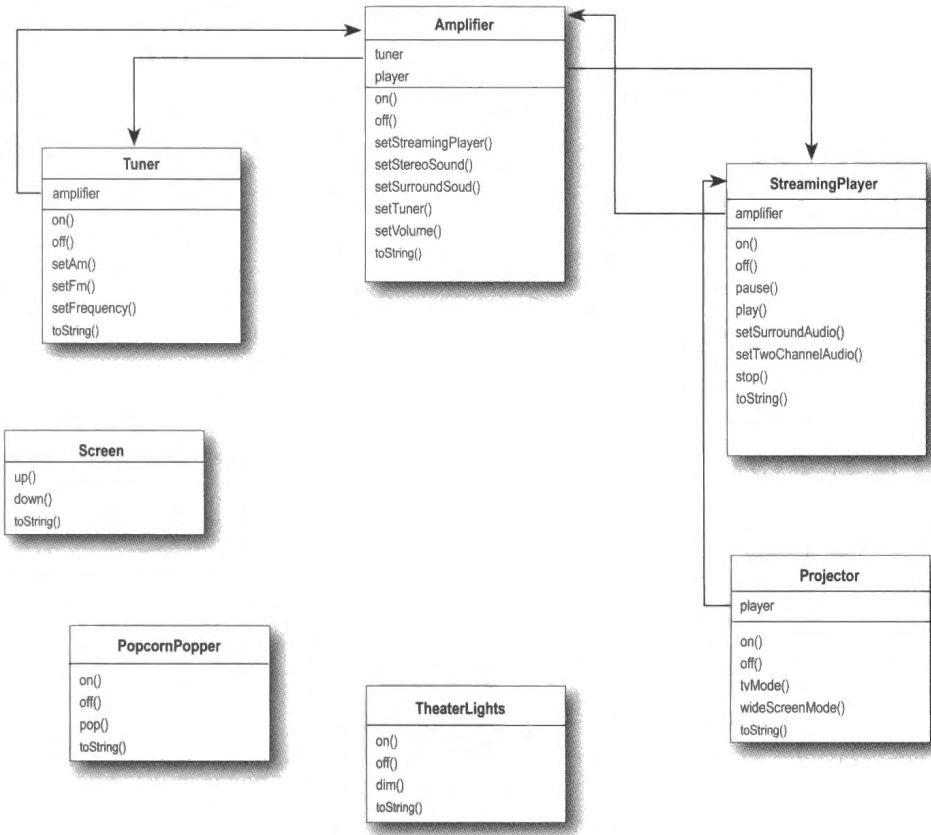
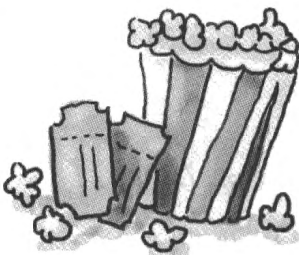
模式	意图
装饰者	转化一个接口为另一个
适配器	不改变接口，但添加责任
外观	使接口更简单

甜蜜的家庭影院

在深入外观模式的细节之前，我们来看一个日益增长的国民热潮：为刷剧看电影和剧集而建造一套吸引人的家庭影院。

你做了研究，组装了一套杀手级的系统，里面有流媒体播放器、投影视频系统、自动屏幕、环绕音响，甚至还有爆米花机。

所有你要整合的组件如下：



有很多类，很多交互，还有一大群接口，等着学习和使用。

你花了好几个星期布线、装投影机、连接所有设备并微调。现在，是时候让所有设备运转起来，享受一部电影了……

观赏电影（困难方式）

挑选一部影片，放松，准备开始感受电影的魔力。噢，还有一件事：要看电影，你需要执行一些任务：

- ❶ 打开爆米花机。
- ❷ 开始爆米花。
- ❸ 调暗灯光。
- ❹ 放下屏幕。
- ❺ 打开投影机。
- ❻ 设置投影机输入为流媒体播放器。
- ❼ 把投影机设置在宽屏模式。
- ❽ 打开功放。
- ❾ 设置功放为流媒体输入。
- ❿ 设置功放为环绕音响。
- ⓫ 设置功放音量为中(5)。
- ⓬ 打开流媒体播放器。
- ⓭ 开始放电影。



我们用类和方法调用的方式执行这些任务：



但还有更多……

- 电影结束后，你怎么把每样东西关掉？难道你不得不反向地把这一切再过一遍？
- 听广播是不是也要这样复杂？
- 如果你决定升级你的系统，你可能不得不学会稍微不同的流程。

那怎么办？使用家庭影院的复杂性大家都看得到！

我们来看，外观模式如何把我们拉出泥潭，让我们享受电影……

灯、相机、外观!

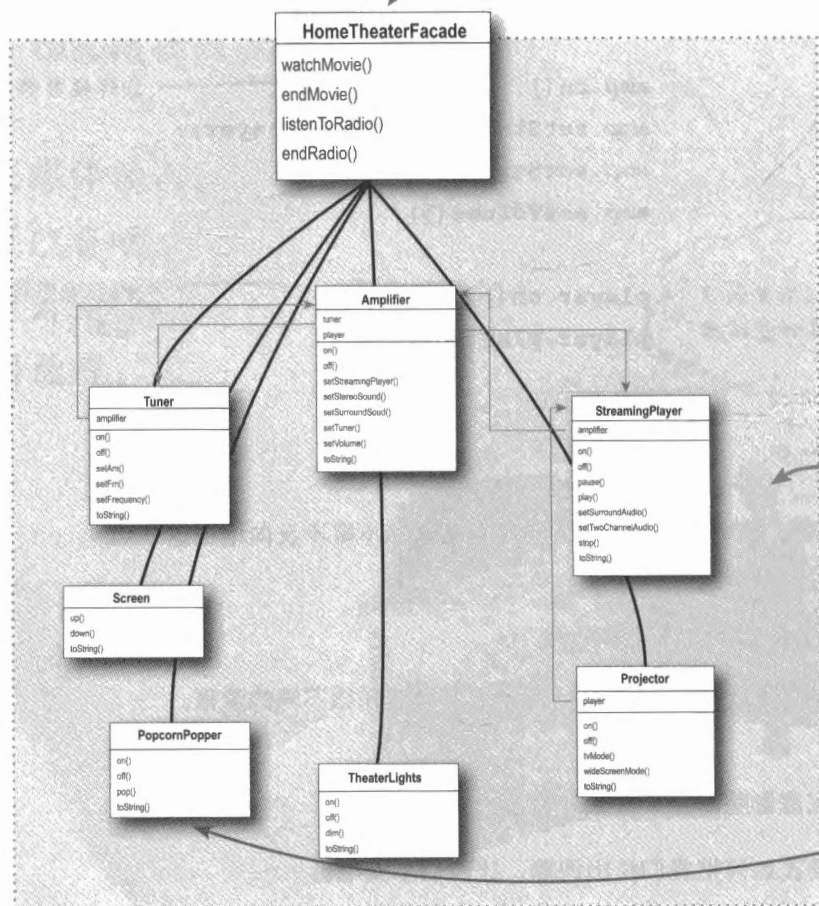
外观正是你需要的：有了外观模式，通过实现提供一个或多个更合理接口的外观类，你可以使一个复杂的子系统变得容易使用。别担心，如果你需要复杂子系统的威力，它还在那里，但如果你需要的是一个直截了当的接口，那就用外观。

我们看看外观如何运作：

- 1 OK，是时候为家庭影院系统创建一个外观了。我们创建一个新类 `HomeTheaterFacade`，它暴露一些简单的方法，例如 `watchMovie()`。

- 2 外观类把家庭影院组件看作子系统，调用子系统来实现 `watchMovie()` 方法。

外观正在简化的子系统。



play()

on()

`watchMovie()`

子系统外观的客户。

- 3 你的客户代码现在调用家庭影院外观的方法，而不是子系统的方法。因此，现在要看电影，我们只要调用一个方法`watchMovie()`，它会和灯、流媒体播放器、投影机、功放、屏幕、爆米花机等沟通。

我一定要能访问
低层的部分！



拉什莫尔高中影音科学俱乐部的前任社长。

- 4 外观依然保持子系统可访问，所以你可以直接使用。如果你需要子系统类的高阶功能，它们还是可以使用的。

没有蠢问题

问：如果外观封装子系统类，需要底层功能的客户如何能访问到它们？

答：外观不“封装”子系统类。它们只是给功能提供一个简化的接口。在客户需要使用更多特定接口时，依然可以直接使用子系统类。这是外观模式一个很棒的特征：它提供一个简化的接口，但依然暴露系统的全部功能给需要的客户。

问：外观添加了任何功能吗，或者它只是把每个请求传递给子系统？

答：除了使用子系统之外，外观可以添加自己认为“聪明”的功能。例如，虽然我们的家庭影院外观没有实现任何新行为，但它足够聪明，知道爆米花机在爆米花之前必须先开机（和如何开机并筹划一次电影上映的细节一样）。

问：是否每个子系统只有一个外观？

答：不必。外观模式肯定允许为给定子系统创建任意数量的外观。

问：除了能够提供一个更简单的接口之外，外观模式还有其他好处吗？

答：外观模式也允许你把客户实现从任何一个子系统解耦。假如说，你获得大幅加薪，决定升级你的家庭影院，所有新组件都有不同的接口。如果之前你的客户代码是针对外观而不是子系统，客户代码不需要变化，只需要修改外观（但愿制造商会提供）！

问：因此，辨别适配器模式和外观模式之间差别的方法是，适配器包装一个类，外观可能代表很多类？

答：不是！记住，适配器模式改变一个或多个类的接口为一个客户期望的接口。虽然大多数教程的例子展示的适配器是适配一个类，但你可能需要适配很多类来提供接口让客户编码。一个外观也可能对单个有非常复杂接口的类提供一个简化的接口。

两者之间的差别不在于“包装”了多少类，而是其意图。适配器模式的意图是修改接口以匹配客户的期望。外观模式的意图是提供子系统的一个简化的接口。

外观不仅简化接口，它还把客户从组件的子系统解耦。

外观和适配器可以包装多个类，但外观的意图是简化，而适配器的意图是转化接口为不同的接口。

构造家庭影院外观

我们来逐步构造HomeTheaterFacade类。第一步是使用组合，以便外观可以访问子系统的所有组件：

```
public class HomeTheaterFacade {
```

```
    Amplifier amp;
```

```
    Tuner tuner;
```

```
    StreamingPlayer player;
```

```
    Projector projector;
```

```
    TheaterLights lights;
```

```
    Screen screen;
```

```
    PopcornPopper popper;
```

这是组合；我们会用到的
子系统组件都在这里。

```
public HomeTheaterFacade(Amplifier amp,
```

```
    Tuner tuner,
```

```
    StreamingPlayer player;
```

```
    Projector projector,
```

```
    Screen screen,
```

```
    TheaterLights lights,
```

```
    PopcornPopper popper) {
```

将子系统中每一个组件的引用
传入外观的构造器。外观把它
们赋值给相应的实例变量。

```
    this.amp = amp;
```

```
    this.tuner = tuner;
```

```
    this.player = player;
```

```
    this.projector = projector;
```

```
    this.screen = screen;
```

```
    this.lights = lights;
```

```
    this.popper = popper;
```

```
}
```

```
// 其他方法放这里
```

这部分的代码，一会就填进去……

```
}
```

实现简化的接口

现在，是时候把子系统的组件整合进一个统一的接口了。我们来实现watchMovie()和endMovie()方法：

```
public void watchMovie(String movie) {
    System.out.println("Get ready to watch a movie...");
    popper.on();
    popper.pop();
    lights.dim(10);
    screen.down();
    projector.on();
    projector.wideScreenMode();
    amp.on();
    amp.setStreamingPlayer(player);
    amp.setSurroundSound();
    amp.setVolume(5);
    player.on();
    player.play(movie);
}
```

watchMovie()遵循我们之前不得不手动来做的相同顺序，但把它包装成做所有工作的更称手的方法。注意，对于每项任务，我们都把责任委托给子系统内的相应组件。

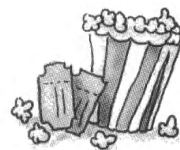
```
public void endMovie() {
    System.out.println("Shutting movie theater down...");
    popper.off();
    lights.on();
    screen.up();
    projector.off();
    amp.off();
    player.stop();
    player.off();
}
```

endMovie()负责关闭一切。同样，每项任务都是委托子系统中合适的组件。



想想看，你在Java API中遇到过的外观。你还希望有哪些新外观？

是时候观赏电影了（容易方式）



演出开始!

```
public class HomeTheaterTestDrive {
    public static void main(String[] args) {
        // 在这里实例化组件
```

在这里，我们就在测试程序中直接创建组件。正常情况下，某个外观会派给客户；客户不需要自己构造外观。

```
        HomeTheaterFacade homeTheater =
            new HomeTheaterFacade(amp, tuner, player,
                                  projector, screen, lights, popper);
```

首先，用子系统中所有的组件来实例化外观。

```
        homeTheater.watchMovie("Raiders of the Lost Ark");
        homeTheater.endMovie();
    }
```

使用简化的接口，先开启电影，然后关闭。

这是输出。调用外观的 watchMovie()，一切搞定……

File Edit Window Help SnakesWhy'dItHaveToBeSnakes?

```
%java HomeTheaterTestDrive
Get ready to watch a movie...
Popcorn Popper on
Popcorn Popper popping popcorn!
Theater Ceiling Lights dimming to 10%
Theater Screen going down
Projector on
Projector in widescreen mode (16x9 aspect ratio)
Amplifier on
Amplifier setting Streaming player to Streaming Player
Amplifier surround sound on (5 speakers, 1 subwoofer)
Amplifier setting volume to 5
Streaming Player on
Streaming Player playing "Raiders of the Lost Ark"
Shutting movie theater down...
Popcorn Popper off
Theater Ceiling Lights on
Theater Screen going up
Projector off
Amplifier off
Streaming Player stopped "Raiders of the Lost Ark"
Streaming Player off
%
```

……在这里，我们已经看完电影，因此调用 endMovie() 将一切关闭。

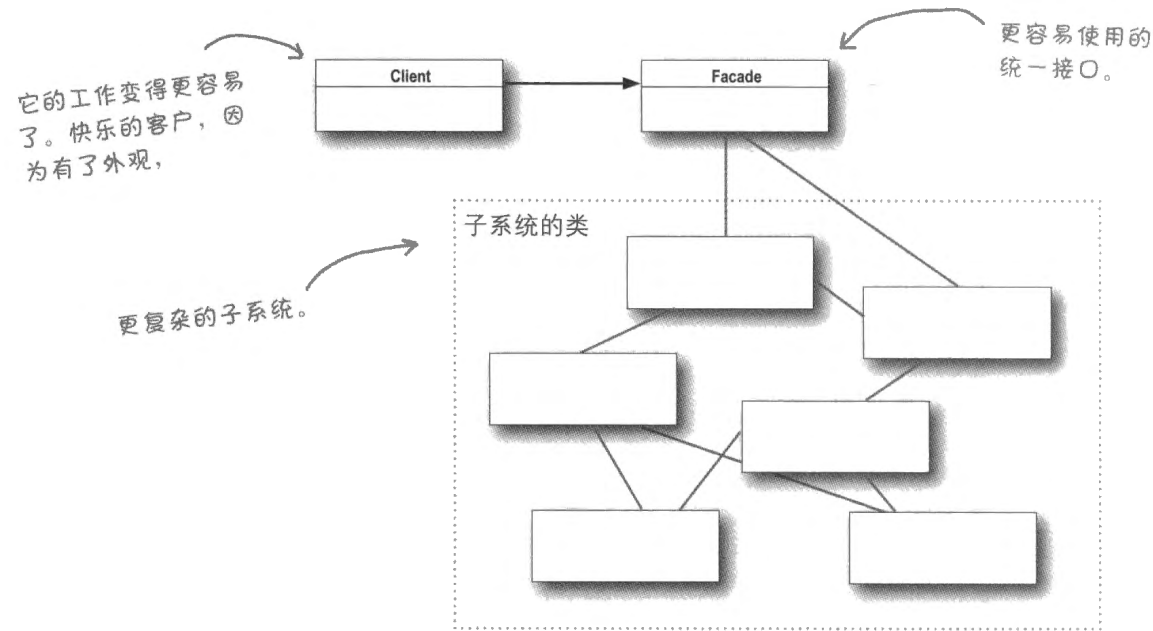
定义外观模式

要使用外观模式，我们创建一个类，这个类把属于某些子系统的一套复杂的类简化和统一。和许多模式不同，外观相当直截了当，没有难以理解的抽象，但这并不会让它的威力降低：外观模式让我们避免客户和子系统之间的紧耦合，还有，正如你马上看到的，它也帮助我们遵守一条新的面向对象原则。

在介绍新的原则之前，我们看看模式的官方定义：

外观模式为子系统的一组接口提供了一个统一的接口。外观定义了一个更高级别的接口，使得子系统更容易使用。

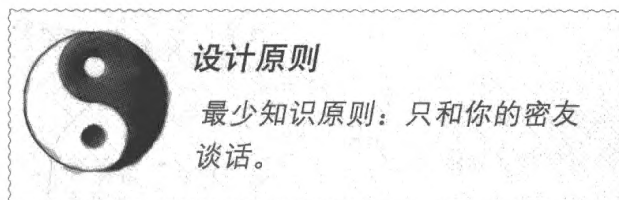
其中的大多数东西你已经知道，但最重要的事情之一是务必记得模式的意图。这个定义清楚明确地告诉我们，外观的目的是通过简化接口使得子系统更容易使用。在模式的类图中，你可以看到这一点：



就这样了。你又学会了一个模式！现在，是时候来看新的OO原则了。小心，这个原则可能有点挑战性！

最少知识原则

最少知识原则引导我们减少对象之间的交互，减少到仅发生在一些亲密“朋友”之间。这个原则通常表述如下：



但这到底是什么意思？意思是，当你在设计一个系统时，对于任何对象，都要注意它所交互的类的数量，以及它和这些类如何交互。

这个原则防止我们创建有大量的类在一起的设计，免得系统一部分的变化会连锁影响到其他部分。当你在许多类之间造成许多依赖时，你的系统就是一个易碎的系统，需要花费许多成本维护，而且复杂得难以让别人理解。



这段代码耦合了多少类？

```
public float getTemp() {  
    return station.getThermometer().getTemperature();  
}
```

如何不赢得朋友和影响对象

OK，但如何不赢得朋友和影响对象呢？该原则提供了一些指南：对于任何对象，从该对象的任何方法，只调用属于以下范围的方法：

- 对象自身。
- 作为参数传给方法的对象。
- 该方法创建或实例化的任何对象。
- 对象的任何组件。

注意，这些指南告诉我们，不要调用从其他方法返回的对象的方法！

把“组件”想成被实例变量引用的任何对象。换句话说，想成是有一个（HAS-A）关系。

这听起来有点严厉，不是吗？调用从另一个调用中返回的对象的方法，有什么害处呢？好吧，如果我们这样做，相当于向另一个对象的子部分发请求（而增加我们直接认识的对象数目）。在这种情况下，原则强制我们，要求该对象为我们做出请求，这样我们就不必知道它的组件对象（保持我们的朋友圈尽可能小）。例如：

不遵守原则

```
public float getTemp() {  
    Thermometer thermometer = station.getThermometer();  
    return thermometer.getTemperature();  
}
```

这里，我们从气象站取得温度计（thermometer）对象，然后自行调用getTemperature()方法。

遵守原则

```
public float getTemp() {  
    return station.getTemperature();  
}
```

当我们应用该原则时，我们给Station（气象站）类添加一个方法，向温度计发请求。这减少了所依赖的类的数目。

保持你的方法调用在界限内……

这是一个Car（汽车）类，展示了你在调用方法同时依然遵守最少知识原则的所有方式：

```
public class Car {
    Engine engine;
    // 其他实例变量
```

这是这个类的一个组件。我们可以调用它的方法。

```
public Car() {
    // 初始化引擎等
}
```

在这里我们创建了一个新的对象；调用它的方法是合法的。

```
public void start(Key key) {
    Doors doors = new Doors();
    boolean authorized = key.turns();
    if (authorized) {
        engine.start();
        updateDashboardDisplay();
        doors.lock();
    }
}
```

你可以调用作为参数传递的对象的方法。

你可以调用对象组件的方法。

你可以调用对象内的局部方法。

你可以调用你创建或实例化的对象的方法。

```
public void updateDashboardDisplay() {
    // 更新显示
}

}
```

没有蠢问题

问：另一个原则叫得墨忒耳法则（Law of Demeter），它和最少知识原则有什么关系？

答：这两个是一个东西，经常换着使用。我们倾向于使用最少知识原则，基于两个原因：①这个名字更直白；②使用“法则（Law）”这个词

暗示我们总是必须应用这个原则。事实上，没有原则是法律（law）；所有原则都应该在有帮助的时间和地点才使用。所有设计都有折中（抽象与速度、空间与时间等），而原则提供指南，在应用它们之前，你应该考虑所有因素。

问：应用最少知识原则有什么缺点？

答：是的，虽然这个原则减少了对象之间的依赖，研究显示这会减少软件维护成本，应用这个原则也会导致编写更多的“包装者”类来处理对其他组件的方法调用。这会造成复杂度和开发时间增加，运行时性能下降。



削尖你的铅笔

这些类有没有违反最少知识原则？为什么？

```
public House {
    WeatherStation station;

    // 其他方法和构造器

    public float getTemp() {
        return station.getThermometer().getTemperature();
    }
}

public House {
    WeatherStation station;

    // 其他方法和构造器

    public float getTemp() {
        Thermometer thermometer = station.getThermometer();
        return getTempHelper(thermometer);
    }

    public float getTempHelper(Thermometer thermometer) {
        return thermometer.getTemperature();
    }
}
```



安全帽区域，
小心落物

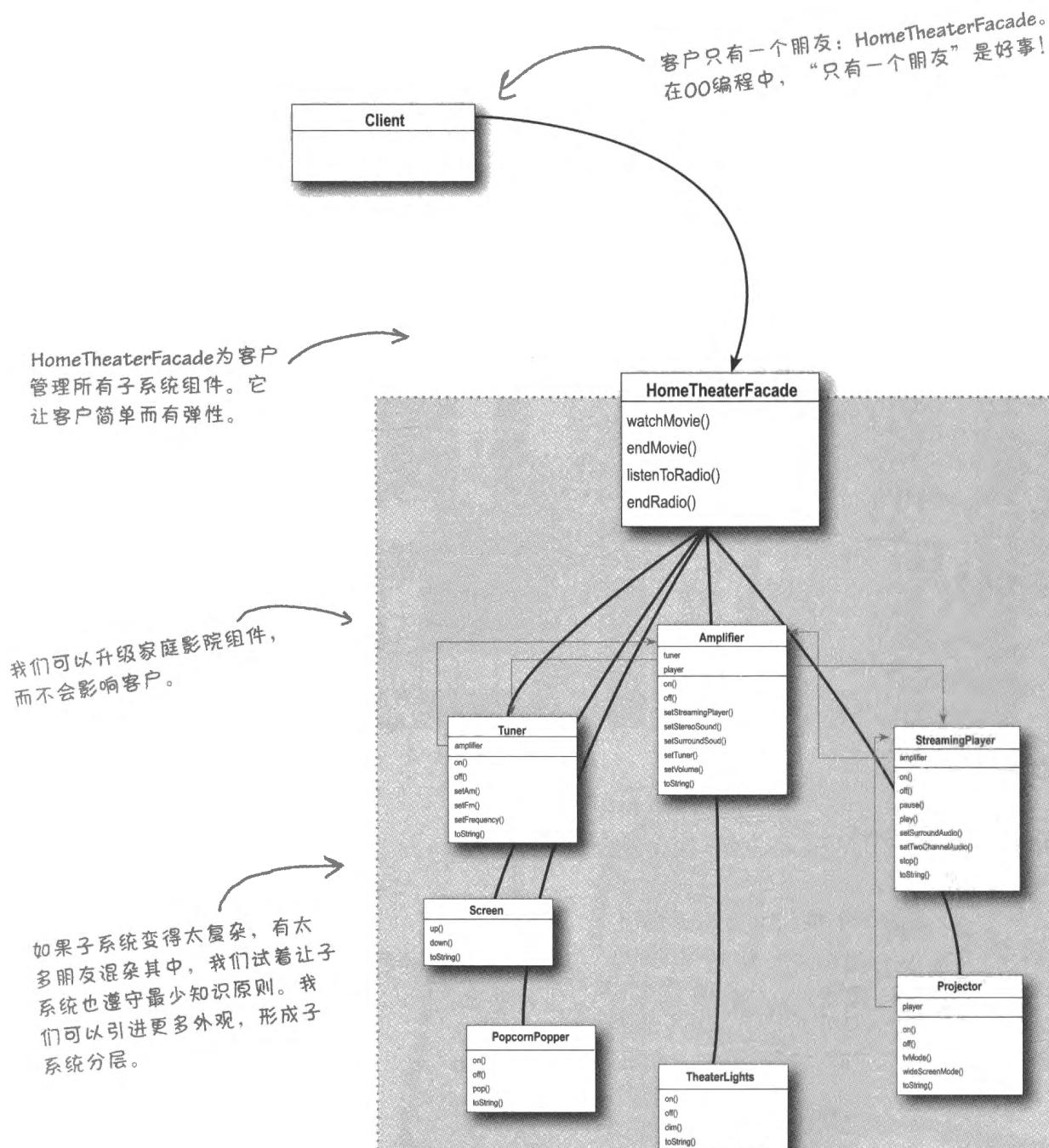


你能够想到Java哪些常用的地方违反了最少知识原则？

你应该在意吗？

回答：System.out.println()如何？

外观模式和最少知识原则





你的设计工具箱中的工具

你的工具箱开始变重了！本章中，我们添加了两个模式，它们允许我们修改接口以及减少客户和所用系统之间的耦合。

OO 基础

OO 原则

封装变化。

组合优于继承。

针对接口编程，而不是针对实现编程。

努力达到交互对象之间的松耦合设计。

类应该对扩展开放，对修改关闭。

依赖抽象，不要依赖具体类。

只和朋友交谈。

抽象

封装

多态

继承

我们有了一个新的技巧，用来使设计中的耦合维持低水平（记住，只和你的朋友说）……

OO Patterns

适配器模式，将一个类的接口转换成客户期望的另一个接口。适配器让原本接口不兼容的类可以合作。

外观模式，为子系统的一组接口提供了一个统一的高级别的接口。外观定义了一个更容易使用。

……还有了两个新模式。它们都改变接口，适配器是转化，外观是统一和简化。

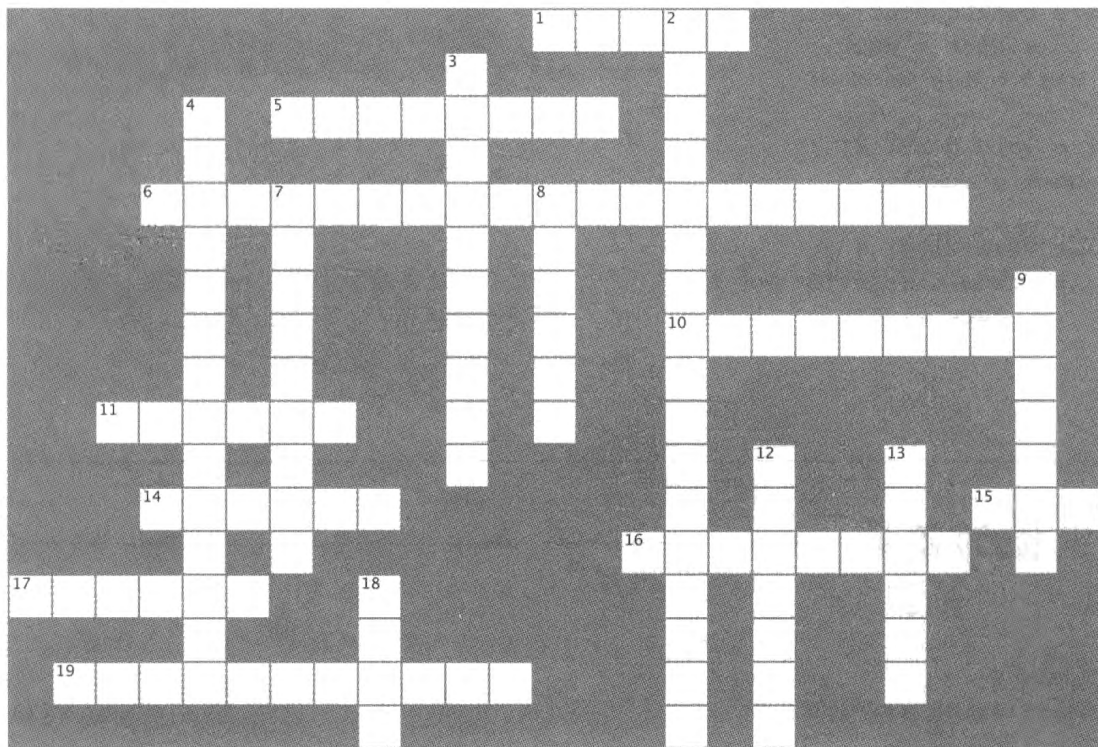
要点

- 当你需要使用一个已有的类，而其接口不符合你的需要，就用适配器。
- 当你需要简化并统一一个大接口，或者一个复杂的接口集，就用外观。
- 适配器改变接口以符合客户的期望。
- 外观将客户从一个复杂子系统解耦。
- 实现一个适配器可能不需要花什么功夫，也可能花很多功夫，视目标接口的大小与复杂度而定。
- 实现一个外观，需要把外观和子系统组合，使用委托来执行外观的工作。
- 适配器模式有两种形式：对象适配器和类适配器。类适配器需要用到多重继承。
- 你可以为一个子系统实现多于一个的外观。
- 适配器包装一个对象以改变其接口，装饰者包装一个对象以添加新的行为和责任，而外观“包装”一群对象以简化其接口。



设计模式填字游戏

是的，又是一个填字游戏。所有答案词汇都在本章中。



横排

1. 真还是假？适配器只能包装一个对象。
5. 一个适配器_____一个接口。
6. 我们看的影片（五个词）。
10. 如果在英国，你可能需要其中之一（两个词）。
11. 带两个角色的适配器（两个词）。
14. 外观依然_____低层访问。
15. 鸭子做这个比火鸡做得好。
16. 最少知识原则的缺点：太多_____。
17. 一个_____简化一个接口。
19. 新的美国梦（两个词）。

竖排

2. 装饰者称呼适配器这个（三个词）。
3. 外观的一个优点
4. 一个原则，没有听起来那么容易（两个词）。
7. 一个添加新行为的_____。
8. 伪装成鸭子。
9. 违反最少知识原则的例子：System.out_____。
12. 没有这个就不能完成影片。
13. 适配器客户使用_____接口。
18. 可以说，适配器和装饰者_____一个对象。



削尖你的铅笔 答案

假设我们也需要一个适配器，将鸭子转成火鸡。我们称它为 DuckAdapter。答案如下：

```
public class DuckAdapter implements Turkey {
    Duck duck;
    Random rand;
    public DuckAdapter(Duck duck) {
        this.duck = duck;
        rand = new Random();
    }
    public void gobble() {
        duck.quack();
    }
    public void fly() {
        if (rand.nextInt(5) == 0) {
            duck.fly();
        }
    }
}
```

现在我们把Turkey适配到Duck，因此我们实现Turkey接口。

我们存储一个到正在适配的Duck的引用。

我们也创建一个随机数对象，在fly()方法中会看到它如何使用。

咯咯叫变成呱呱叫。

因为鸭子比火鸡飞得更远，我们决定让鸭子平均五次只飞一次。



削尖你的铅笔 答案

这些类有没有违反最少知识原则？为什么？

```
public House {
    WeatherStation station;
    // 其他方法和构造器
    public float getTemp() {
        return station.getThermometer().getTemperature();
    }
}

public House {
    WeatherStation station;
    // 其他方法和构造器
    public float getTemp() {
        Thermometer thermometer = station.getThermometer();
        return getTempHelper(thermometer);
    }

    public float getTempHelper(Thermometer thermometer) {
        return thermometer.getTemperature();
    }
}
```

违反最少知识原则！你在调用由另一次调用的返回的对象的方法。

没有违反最少知识原则！看起来像是略施小计绕过了原则。这样做真的有什么变化吗？因为我们只是把调用移到另一个方法。



练习答案

你已经看到如何实现一个适配器，适配Enumeration到Iterator。现在，写一个适配器，适配Iterator到Enumeration。

```
public class IteratorEnumeration implements Enumeration<Object> {
    Iterator<?> iterator;

    public IteratorEnumeration(Iterator<?> iterator) {
        this.iterator = iterator;
    }

    public boolean hasMoreElements() {
        return iterator.hasNext();
    }

    public Object nextElement() {
        return iterator.next();
    }
}
```

注意，我们保持类型参数为泛型，因此这对所有对象类型都适用。

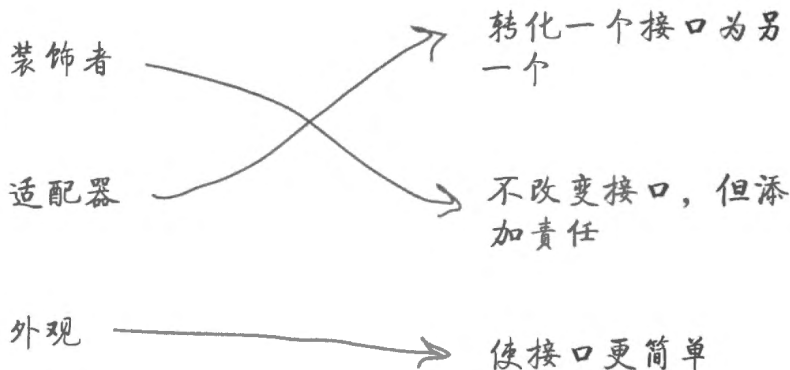
WHO DOES WHAT?

答案

请将下列模式和描述配对：

模式

Intent





设计模式填字游戏解答

